

2. Three kinds of “model” (why you got lost)

Audience: Charles, re-entering
Standalone: definitions are inline; no other files required.
Slide summary (same story): `Model.pdf`

The mistake that feels like failure but is not

You kept hearing “the model” and assumed one equation should do everything: fit the hero, explain NBA psychology, simulate a league, and satisfy Alex — all at once.

This project actually uses **three different objects**. They stack, but they are **not the same thing**. Confusion came from switching layers mid-sentence — not from you being unable to do math.

BINDING insight — selection is its own step (#1 confusion)

Charles lock (Jul 2026). Full binding doc: `../BINDING_Selection_is_its_own_step.md`.

You were merging two different mechanisms:

- 1. **Environment / development** — peers affect you through benefit (B) and congestion (D). The **combined environment object** is `L_net = B - D`. This **describes the peer environment**, not who wins the slot.
- 2. **Selection** — a **separate step**: who actually gets the scarce advancement (draft pick). **Alex’s equation is this step only:** `S_i = A_i - λ · L_C` — own ability minus congestion penalty among viable peers in **who gets selected**.

Environment (<code>L_net</code> = B - D)		Selection (<code>S_i</code> = Alex score)
Asks	How do peers help vs hurt development?	Who gets picked?
Hero plot	Does not separate these channels	Shows outcome only (draft rate vs pool quality)
Sim (538D)	Who lands on which roster	Rank → top K using a selection rule

Carry this sentence: *The hero describes outcomes; `L_net` is the net peer environment (B - D); `S_i` is who gets selected; the sim tests whether congestion in the **selection rule** changes who gets selected.*

Layer C is powerful because it **names selection as a step** — talent-only vs congestion-in-score — instead of hiding selection inside a giant environment model.

Layer A — describe the curve (phenomenology)

Job: Answer “What does the data look like?”

Tools: Bin the real panel. Optionally fit a simple curve (e.g. draft probability vs pool quality plus its square).

Basketball example: The hero plot and a quadratic regression on the same filtered rows.

We fit a **quadratic** (not a line) because the hero bends at the top — it is the simplest way to ask whether draft rate is **concave** in pool quality (negative squared term) and to draw a smooth overlay on the bins. We are **not** claiming NBA teams use that formula; the **binned hero** stays the headline stylized fact.

What success looks like: Negative curvature — draft rate bends down at the top bins — on the **locked** hero definition (16 bins, leave-one-out quality, minutes filter, seasons 2011–2021).

What this does NOT prove:

- Why the curve exists.
- That NBA teams use our formula.
- That simulation must match bin-by-bin.

Analogy: Layer A is a **weather report**. It records the shape of the cloud. It is not a **theory of rain**.

Layer B — explain help vs hurt (mechanism story)

Job: Answer “Why could good peers both help and hurt?” — and **separate environment from selection**.

Tools: Words and symbols — not one estimated equation in v1.

Part 1 — Environment (B minus D)

Net value from the peer environment splits into two parts that **pull in opposite directions**:

- **B (benefit)** — upside from good peers: development, visibility, playing in a strong context.
- **D (congestion / dilution)** — downside from good peers: harder to stand out, more competition around you.

Write this as $L_{net} \approx B - D$ (net local environment). As pool quality rises, **both** B and D can increase. Early on, benefit may dominate (draft rate rises). At the very top, **congestion can catch up or outweigh benefit** — a story for why the hero tail **might** dip. In v1 we are **not** estimating separate B and D curves from data.

This part describes the environment — not who gets drafted.

Part 2 — Selection (Alex score — its own step)

Selection is separate. After rosters exist, someone must decide **who gets the scarce slot** (NBA draft pick).

Alex’s equation is about that decision rule, not the full environment:

- $S_i = A_i - \lambda \cdot L_C$ — own **ability** minus **weight × viable-peer congestion in selection** (equivalently $S_i = A_i + \lambda(B-D)$ with $(B-D) = -L_C$ in the score for Alex v1)
- L_C — **D-ish** congestion measured **LOO on a roster** after assignment; not full L_{net} , not $L_C = f(B, D, \tau)$ in v1

That is why the simulation makes **selection explicit**: assign players → compute **selection score** → take top K. **Knockout** (repo shorthand — say **mechanism contrast** to Alex if you prefer): hold the fake league fixed and **remove congestion from the selection score**. Compare top-K on $S_i = A_i$ only ($\lambda = 0$) vs top-K on $S_i = A_i - \lambda \cdot L_C$. The **one ingredient toggled** is L_C **in selection** — not talent, not the whole L_{net} environment. We plot both curves side by side, but the logic is **turn that term off, see if the curve changes**, not two unrelated “versions” of the sim.

What success looks like: The story is coherent; sim shows ability-only **fails** the congestion-in-selection story when $\lambda = 0$ or congestion is dropped from the score.

What this does NOT prove in v1:

- Separate measurement of B(Q) and D(Q) on one axis.
- That real NBA front offices compute Alex’s score literally.
- That the hero curve proves selection channel only (hero is outcome; channel is theory + sim).

In plain terms: Layer B Part 1 = environment story (B – D). Layer B Part 2 = **selection rule** (Alex). Layer C = **run the selection step in code**.

Unified nesting — how the pieces fit (beyond v1)

One way to **nest** the symbols without merging layers:

General selection score: $S_i = A_i + \lambda \cdot (B - D)$ (ability plus weighted net local environment in the **selection rule**).

Restriction	Meaning	Score
Knockout	$\lambda = 0$ — remove all of (B – D) from selection	$S_i = A_i$
Alex v1	Only congestion enters selection: (B – D) = –L_C in the score (not “B is zero in the world”)	$S_i = A_i - \lambda \cdot L_C$
Full (later)	$\lambda \neq 0$ with both B and D in (B – D)	Richer; not what v1 runs

Full generative pipeline (Layer C — two steps for a minimal POC):

Draw A_i → assign to rosters (ρ = assignment assortativity) → compute L00 poolq_loo, L_C on rosters
→ $S_i = A_i + \lambda(B-D)$ [v1: (B–D)=–L_C] → top K → bin for plots

Knob	Step	Role
ρ (rho)	Assignment (who lands where)	Assortativity in soft match to team targets T_j ; $\rho=0$ = max mixing; $\rho \uparrow$ = sharper match; not inside S_i
λ	Selection	Weight on $(B - D)$ in the score; v1 knocks out via $\lambda = 0$ or sets $(B - D) = -L_C$
B, D	Environment (theory)	$L_{net} = B - D$ — help vs hurt among peers; not separately estimated in v1
L_C	Computed on rosters after assignment	D-ish operational congestion (LOO viable-peer crowding, e.g. $crowding_smooth$) — not the same object as full L_{net} , and not a clean formula $L_C = f(B, D, \rho)$

Minimal generative claim (Alex v1): some **roster step** (so LOO stats exist) **and** a **score** $\rightarrow$ **top-K step** (congestion in vs out of S_i). **Calibrating ρ** to 530 or matching hero bins is **not** the v1 gate.

Hero mapping (stay honest): The hero is **Layer A outcome** — mean draft rate by $poolq_loo$ on the **real panel**. It **motivates** the story; it does **not** identify (ρ, λ, B, D) or prove S_i is the NBA rule. $poolq_loo$ is a **pool proxy**, not measured $(B - D)$.

L_C and θ (**viability cutline**): On a roster, L_C = LOO congestion from **viable peers**. θ = viability threshold (530 default: $med(perf \mid \text{ever drafted})$). **Hard:** count teammates with $A_j > \theta$. **Smooth (538D default):** LOO mean of $\sigma(y(A_j - \theta))$ ($crowding_smooth$). y = sharpness of the sigmoid around θ .

Optional follow-up (Alex asked Jul 2026): ρ sim ablations — fix selection rule, vary assignment assortativity — test “is sorting involved?” **Not** required for v1 minimal model; the headline knockout is $\lambda = 0$ vs $(B-D)=-L_C$ in S_i (already run; see doc 03).

Layer C — simulate a fake league (generative proof-of-concept)

Job: Answer “If we write explicit draft rules, does congestion in the score change who gets picked?”

Tools: Simulation notebook (538D path): create synthetic players $\rightarrow$ assign to teams $\rightarrow$ rank by a **selection score** $\rightarrow$ draft top K $\rightarrow$ plot draft rate by bins.

Headline comparison (knockout on L_C in selection):

- Knockout arm ($\lambda = 0$):** $S_i = A_i$ — congestion **removed** from the selection score.
- Full rule for this test:** $S_i = A_i - \lambda \cdot L_C$ — same league, congestion **in** the score.

What success looks like: The two rules produce **different shapes** — especially, talent-only fails to show the “congestion story” (e.g. elite bin behaves differently when congestion enters).

What this does NOT prove:

- Point-for-point match to every hero bin.

- That the real draft **is** this simulation.

Analogy: Layer C is a **wind-tunnel model**. It shows a mechanism **can** produce bent curves. It is not claiming the wind tunnel **is** the sky.

How the three layers stack (one picture in words)

```
Real NCAA data → Layer A → "Here is the hero curve (outcome only)"
      ↓
Story in prose → Layer B → "Environment: L_net (B-D); Selection: S_i; unified nest (B-D)=L_C in score for v1"
      ↓
Fake league code → Layer C → "Assign (ρ) → score (λ, L_C) → top-K; knockout λ=0"
      ↓
Alex slide      → Model.pdf + hero PNG beside sim PNG + limitation sentence
```

You do **Layer A** first (mostly done). You need **Layer C minimal** for the simplified model deadline. **Layer B** has two parts — environment (B – D) and **selection as its own step** (Alex) — you are not failing if you have not “estimated Layer B” as one formula.

What Alex side-by-side actually means (v1)

Not: “Sim reproduces hero exactly.”

Yes:

- Same **kind** of Y-axis (advancement / draft rate).
- **Honest** X-axis labels (empirical leave-one-out quality vs sim bins on the best matching axis you have).
- One clear **limitation sentence**: we show congestion in the score can bend selection; we do **not** claim bin-for-bin replication yet.

That is enough for the simplified model chapter of work.

Notation landmines (same symbol, different day)

While re-entering, ignore cross-domain notation. For **basketball v1 only**, use this cheat sheet:

Symbol / term	Meaning here	Ignore for now
<code>L_net</code>	Net peer environment = B – D (help minus hurt among peers)	Any other “L” in old emails
<code>S_i</code>	Selection score (Alex) — who gets the slot	Hero regression coefficients
<code>L_C</code>	LOO viable-peer congestion on a roster (used in <code>S_i</code>); D-ish, not full <code>L_net</code>	Network centrality papers
θ (theta)	Viability outline for “substitutable peer” — e.g. med(A drafted)	Generic threshold in other domains
γ (gamma)	Sharpness of <code>o(γ(A_j – θ))</code> in smooth <code>L_C</code>	—
ρ (rho)	Assignment assortativity — soft match to team targets T_j ; $\rho=0$ = max mixing; not inside <code>S_i</code>	Legacy τ (temperature) in archived docs (opposite intuition)
Knockout	Same generative league; <code>L_C</code> removed from selection ($\lambda = 0$) vs Alex score with it in	“Mechanism contrast” to Alex
BINDING	Locked rule: <code>L_net</code> $\neq$ <code>S_i</code> ; hero $\neq$ selection equation	—
Pool quality (LOO)	Leave-one-out teammate quality	Army “poolq” without LOO
Ability (<code>A_i</code>)	Player talent in sim / own perf in data	Tenure “productivity”
Lambda (λ)	Weight on <code>L_C</code> in selection score	Global “slot capacity” $\wedge$ across orgs
Hero	Empirical binned draft plot	Any other “hero” in old emails

When a doc uses shorthand without re-defining, **close it** and come back to this table.

What to tell yourself when reading old memos

If a one-page memo assumes you already know which layer is which, it will feel like gibberish. That is a **format mismatch**, not missing intelligence.

While lost: read only re_entry docs 01–03.

When steady: optional short memos become useful again.

Next: [03_Three_Day_Basketball_Focus.md](#)